

Közlekedési objektumok detektálása YOLOv5-ös hálóval

Nyilas Péter Konzulensek: Dr. Hullám Gábor és Révy Gábor

2023. december 15.

1. Bevezetés

Az AI alapú képfelismerést a konvolúciós neurális hálók forradalmasították és széles körben alkalmazzák őket a mai napig különböző objektumfelismerési és osztályozási problémák megoldására.

A konvolúciós neurális hálók speciális rétegei között találhatók az ún. konvolúciós rétegek (Conv), amelyek az input képet szűrőkkel (kernelekkel) dolgozzák fel. Ezek a szűrők olyan jellegzetességeket keresnek a képen, például élek vagy színek, és a kép területi szintű információit vonják ki. Azután a háló további rétegei, például a pooling rétegek (Pool) és teljesen összekapcsolt rétegek, tovább finomítják és kinyerik az absztrakt jellemzőket és továbbítják a következő rétegnek.

A konvolúciós neurális hálók alkalmazása kiterjed a képfelismerésen túl, beleértve az arckifejezések és érzelmek felismerését, Viselkedés analízisen keresztül az önvezető autók látási rendszeréig. Ezek a hálózatok effektíven képesek megtanulni és reprezentálni a vizuális információt.

Én is egy ilyen problémának a megoldásához használtam ezeket a hálókat, pontosabban közlekedési objektumok felismerésére és osztályozására, Dashcam (fedélzeti kamerás) felvételekből. Mely fontos az önvezető autók vagy a vezetést támogató rendszerek fejlesztésében, a mérés technika fejlesztésére mérés elemzési feladatok gyorsítására.

1.1. Felvetett problémakör

A dashcam felvételeket általában ADAS (fejlett vezetéstámogató rendszer) fejlesztésénél gyakran vesznek fel radar és lidar, vagy akár videószenzor validálására egy mérés elemzés közben, ezt manuálisan, irodai dolgozók elemzik és dokumentálják. Ezt gyorsítandó a megoldásom aminél ezeket az objektumokat előre szegmentáljuk, így gyorsítva és könnyítve ezt a feladatot, azzal hogy előre felcímkézzük őket.

2. Konvolúciós Neurális hálók

Az alábbiakban egy konvolúciós háló fontos és rá különösen jellemző építőelemeit mutatom be.

2.1. 2D diszkrét Konvolúció

A 2D konvolúció a neurális hálókban egy olyan művelet, amelyet az input tenzoron (ebben az esetben egy képen) hajtanak végre egy kernel (vagy szűrő) segítségével. A kernel egy kis méretű súlyozott mátrix, amelyet végigmozgatnak az input tenzoron, és a súlyozott összegeket számolják ki a megfelelő részeknél (lásd [Ábra 1](#)). Ezáltal a konvolúció révén újabb tenzor (feature map vagy jellemzőtérkép) keletkezik, amely tartalmazza az input tenzor különböző jellemzőit. Számolása az alábbi módon történik:

- **I**: Az input tenzor (kép).
- **K**: Kernel (szűrő) mátrix.
- **O**: output tenzor (feature map).
- **(i,j)**: tenzorindexek, input és output tenzorok cellapozíciói.

$$O[i, j] = \sum_m \sum_n I[i + m, i + n] \cdot K[m, n]$$

A O tenzor első két dimenziója a pixel (vagy annál már nagyobb egység a képen), a harmadik pedig a pixel színcsatornái (általában 3). Egy konvolúciós réteg olyan egységeket tartalmaz, amelyek mezői az előző réteg egy-egy területét (2x2, 3x3, 5x5) fedik le. Az ilyen egység súlyvektorát (az adaptív paraméterek halmazát) gyakran nevezik szűrőnek.

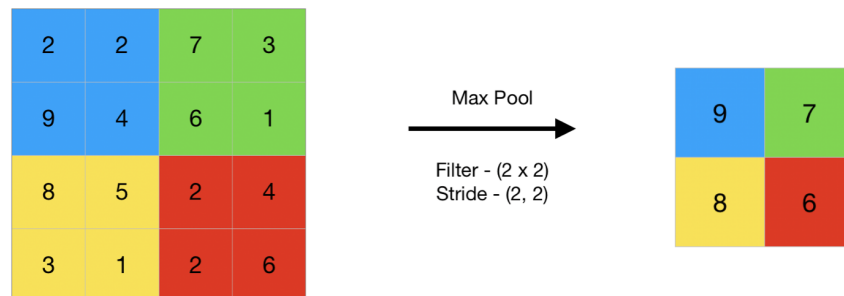
3 ₀	3 ₁	2 ₂	1	0
0 ₂	0 ₂	1 ₀	3	1
3 ₀	1 ₁	2 ₂	2	3
2	0	0	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

1. ábra. Mátrix és kernel

2.2. Pooling

A Pooling (a [Ábra 2](#) képen) réteget a bemenet térbeli dimenzióinak (szélességének és magasságának) csökkentésére használják, miközben megőrzik a mélységet (azaz a csatornák számát). Leggyakoribb verziója a Max pooling, ami előre meghatározott területen számol egy a területen belüli maximum értéket az elemeken és a következő, kisebb dimenziószámú mátrix értéke lesz



2. ábra. MaxPooling

2.3. Teljesen összekapcsolt réteg

Teljesen kapcsolt réteg (Fully Connected, Dense, Linear Combination): előállítja a bemenetek és egy tárolt súlymátrix lineáris kombinációját: $H = XW + b$, ahol X a bemeneti mátrix, W a súlymátrix, b egy opcionális eltolósúly-vektor.

2.4. Batch normalizálás

A batch normalizációs rétegek segítik a túltanulás megelőzését és normalizált batchek mellett megfigyelhető a háló tanulási sebességének javulása is, ugyanis a Neurális hálók az információt "kötegekben" vagyis batch-ekben dolgozzák fel, így ez a réteg egy köteg átlagát és szórását egységesen normalizálja az alábbi módon:

$$\text{Átlag: } atlag = \frac{1}{m} \sum_{i=1}^m x_i$$

$$\text{Szórás: } szoras = \frac{1}{m} \sum_{i=1}^m (x_i - atlag)^2$$

$$\text{Normalizált aktivációk: } y_i = (x_i - atlag) \sqrt{szoras + \epsilon}$$

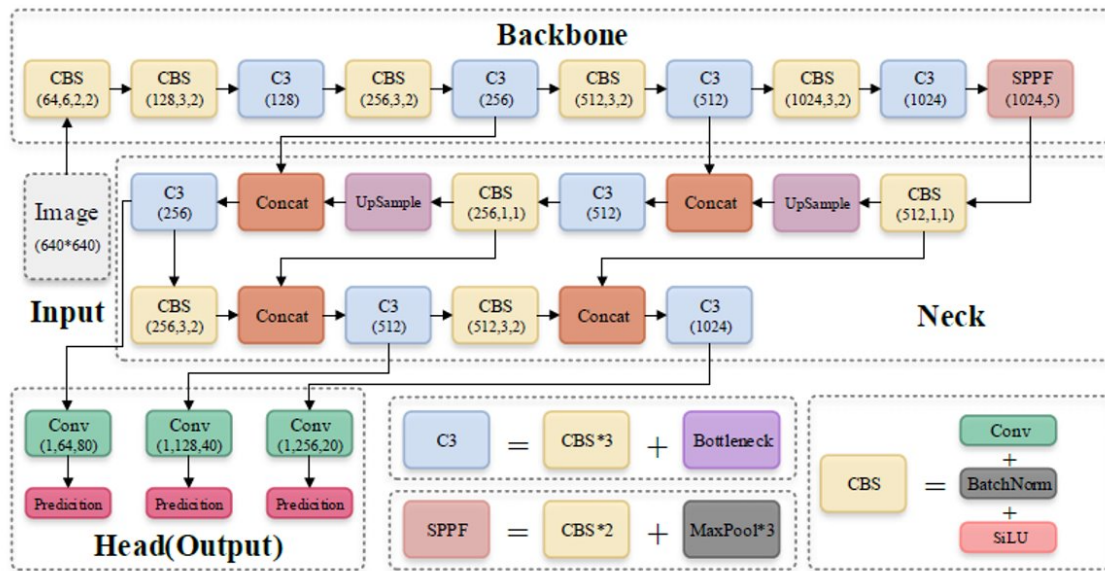
$$\text{Átskálázott és eltoló aktivációk: } z_i = \gamma y_i + \beta, \text{ ahol } \gamma \text{ és } \beta \text{ tanult paraméterek}$$

3. YOLOv5 és gyakorlati alkalmazása

3.1. YOLOv5 és használt verziók

A Yolo(You only look once) algoritmus egy Joseph Redmon, Santosh Divvala, Ross Girshick és Ali Farhadi által kitalált algoritmuson alapuló neurális háló amiből a félév során a YOLOv5-öt (architektúra: [Ábra 3](#)) használtam amit az Ultralytics készített. A Yolo egy egészen új megközelítés a computer vision világában. Működés közben a képet rácsrendszerre osztja, és minden egyes rács a saját területén lévő objektumokat érzékeli. Innen is ered a név, egy lépésben végzi az objektumdetekciót és a lokalizációt is, tehát "Egyszer néz rá".

A háló tanító bemenetére elvárja a képet és egy txt szövegfájlt amiben a képen található objektumok találhatóak id, középpont, szélesség, és hosszúság, szerint.



3. ábra. YOLOv5 architecture from Ultralytics github

A továbbiakban a YOLOv5-öt használtam az Ultralytics github repository-jából töltöttem le:

<https://github.com/ultralytics/yolov5>

A YOLOv5 [2] hálónak öt ismert verziója van, amik méretükben és így hatékonyságukban térnek el egymástól:

1. YOLOv5n (nano) 1,9 millió paraméter,
2. YOLOv5s (small) 7,2 millió paraméter.
3. YOLOv5m (medium) 21,2 millió paraméter.
4. YOLOv5l (large) 46,5 millió paraméter.
5. YOLOv5x (extra large) 86,7 millió paraméter.

Ezek közül hardveres megkötések miatt a YOLOv5s-et választottam, ami bár alulteljesít a nagyobb hálóknak, de a céljainknak megfelel, hiszen ez a háló bár előtanított a COCO adathalmazon (ez egy sokosztályos adathalmaz, nem feltétlenül közlekedési objektumokra, 80 objektuma között állatok és hétköznapi tárgyak) rendelkezik egy valamilyen szintű kompetenciával a közlekedési objektumok detektálására.

3.2. Adathalmaz, Cityscapes

Ezt a hálót tanítottam felül hogy az általam kitűzött cél elérésére, a közlekedési objektumok detektálására, erre használtam fel a "Cityscapes dataset"[1]-et.

Hogy a hálónk effektívebb legyen csak 8 osztályt hagytam meg abból inkább 6 osztály észlelésére tanítottam felül, 8 osztály az eredeti adathalmazból (*gyalogos, autó, biciklis, motoros, busz, vonat, kamion, ismeretlen*) de a Cityscapes-ben csak ebből 6 található (*gyalogos, autó biciklis, motoros, busz, kamion*), erre felül tanítottam, a másik adathalmaz a **CityScapes** annak is a "*gtfine*" nevű finoman annotált adatbázisával, aminek az osztályai, és a COCO adathalmaz osztályainak a metszete hat osztály (*gyalogos, autó, biciklis, motoros, kamion, busz*). A finom annotáció ebben a helyzetben azt jelenti hogy az objektumokat egymástól elszegmentálták és a 'corse' annotációhoz képest lényegesen több pontot használtak poligonok leírásához, amik a szegmentációs maszkokat írják le. Az adatbázis annotált és nyers képeket tartalmaz közúti/városi helyzetekről, több évszakban (tavasz, nyár, ősz), változó fényviszonyok között és különböző időjárási viszonyok között lett felvéve egy fedélzeti kamera (dashcam) szemszögéből.

Az szegmentációs maszkokat minden képhez egy json fájl tárolja, ezekben az objektumok poligonokként, azok pontjaival vannak leírva (és a pontok x és y pozíciójukkal megadva, ezek esetén a mértékegységet a kép adott tengelyen való kiterjedésében adták meg), és egy el vannak látva egy id-vel ami az objektum osztályát hordozza információként.

3.3. Formátumkonverzió

Az előbbi bekezdésekből megérthettük hogy az adathalmaz és a háló bemenete nem ugyanazon a formátumon van (poligon $\neq$ bounding box), ezért írni kellett egy átalakítót ami ezt az átalakítást, gyorsan és megbízhatóan elvégzi, erre egy egyszerű algoritmust írtam.

```
for v in "poligon pontjai":
```

```
    maxx=max(v.x,maxx)    //négyzet legjobboldalibb pontja
```

```
    minx=min(v.x,minx)    //négyzet legbaloldalibb pontja
```

```
    maxy=max(v.y,maxy)    //négyzet legfelso pontja
```

```
    miny=min(v.y,miny)    //négyzet legalsó pontja
```

Ezekből a pontokból csinálunk egy téglalapot az alábbi módon: A téglap középpontját úgy határozzuk meg hogy a

```
for v in "poligon pontjai":
```

```
    width=maxx-minx    //szélesség két szélso x koordináta különbsége
```

```
    height=maxy-miny    //magasság a két szélso y koordináta különbsége
```

```
    centerx=(maxx+minx)/2    //a középpont a két szélso
```

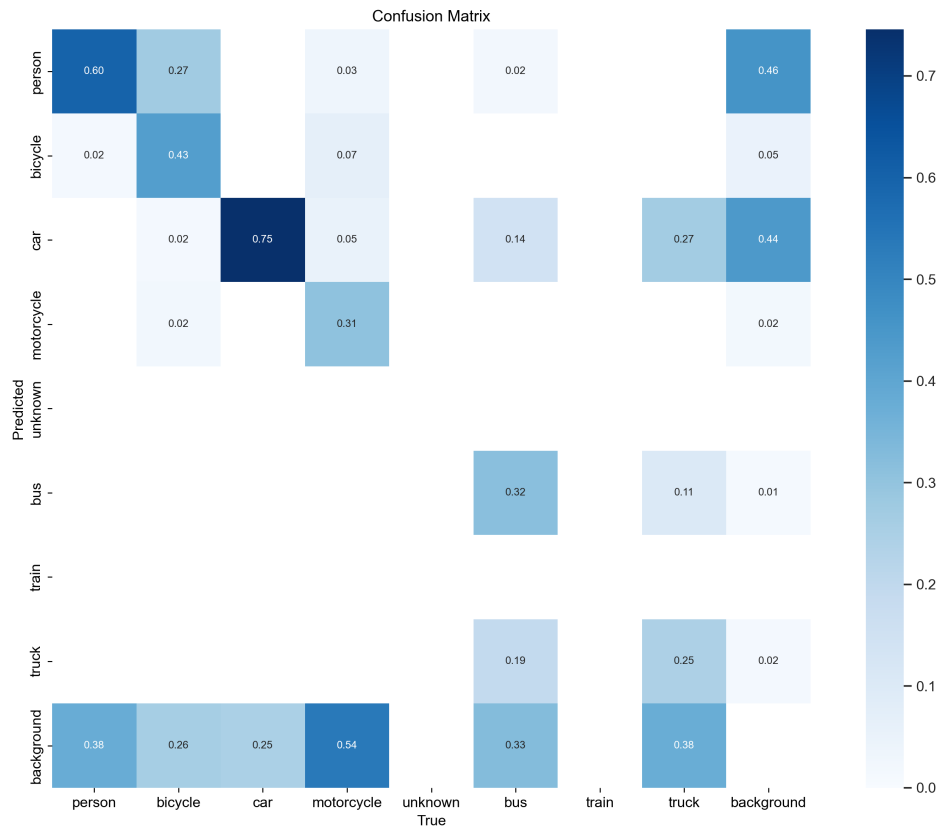
```
    centery=(maxy+miny)/2    //koordinátpáros átlaga
```

3.4. Tanítás és validáció, adatok kiértékelése

A YOLOv5s modellt felül tanítottam a már hálónak megfelelő formátumra, írtam egy gyors adathalmaz leíró '.yaml' kiterjesztésű fájlt az adathalmaz helyéről és megkezdtem a tanítást.

```
$ python train.py --img 1024 --epochs 10 --data Traffic.yaml
    --weights yolov5s.pt --batch-size 4
```

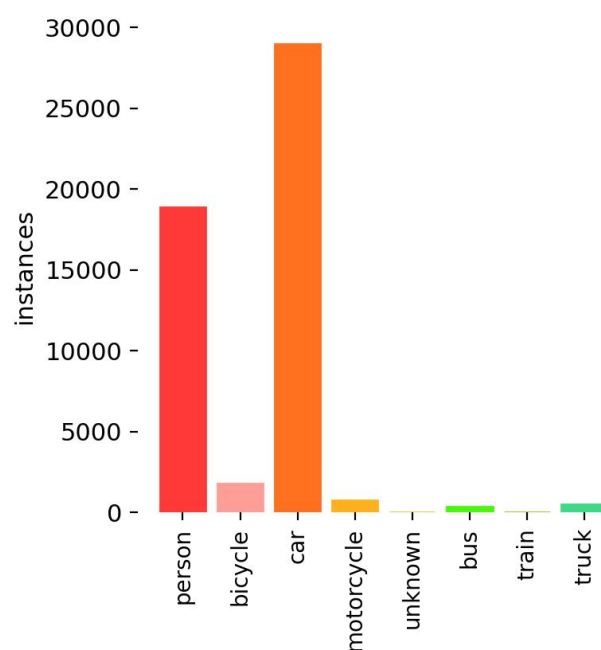
Tehát a tanítást a yolov5s.pt-n végeztem 1024 pixeles felbontáson, 10 epochon keresztül végeztem, és egyszerre a háló 4 képet kapott, ezt írja le a fenti parancs. A tanítás 13 órát vett igénybe és sikerrel zárult, a yolov5 háló elkészítette a validáció eredményeit bemutató infografikákat: [Ábra 4](#)



4. ábra. Confusion matrix

A confusion matrix kiértékelésekor azt tapasztaltam hogy az autókra is gyalogosokra kimondottan érzékeny a háló, ahhoz hogy ezt megértsük látnunk kell a tanító adathalmazt és azon belül azt hogy milyen a tanító adathalmaz osztályainak eloszlása.

Ahogy láthatjuk a gyalogosok és az autók túlnyomó többségben vannak így túlreprezentáltak és eltorzítják az érzékenységet a hálónak az irányukba.



5. ábra. Labelek eloszlása

3.5. Megoldás

A problémát az érzékenységi egyensúly megtalálása jelenti. Mivel a gyalogosok és autók túlnyomó többségben vannak, a YOLO háló érzékenyebbé válik rájuk. A megoldás lehet a tanító adathalmaz újraegyensúlyozása.

Az egyensúlyhiány kiküszöböléséhez több módszer is alkalmazható:

1. **Adatmódosítás:** Több képet kell beszerezni olyan osztályokról, amelyek alulreprezentáltak (például buszok), hogy a háló ne favorizálja a felülreprezentált osztályokat.

2. **Működési módok finomhangolása:** Finomhangolhatjuk a YOLO paramétereit, például az osztályok súlyait, a példák súlyozását és a veszteségfüggvény paramétereit.

3. **Nagyobb hálózat:** Nagyobb YOLO verziókra tanítás (pl. YOLOv5m, YOLOv5l, YOLOv5x), melyek több paraméterrel rendelkeznek és lehet, hogy jobb eredményeket érünk el használatukkal.

Ábrák jegyzéke

1.	Mátrix és kernel	2
2.	MaxPooling	2
3.	Yolov5 architecture from Ultralytics github	3
4.	Confusion matrix	5
5.	Labelek eloszlása	5

Hivatkozások

- [1] C. Dataset, „Cityscapes dataset,” <https://www.cityscapes-dataset.com>, accessed: 2023. december 15..
- [2] Ultralytics, „Yolov5 github repository,” <https://github.com/ultralytics/yolov5>, accessed: 2023. december 15..